Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department	FED
Subject	FEDF, 25CS1201E
Academic Year	I-III Semester (2025-26)
Faculty Name	Dr.Lakshmi Prasanna
Student Name	Rasagnya.M
Roll Number	2520090018
Student Name	Panapakam Hasini
Roll Number	2520030282

AeroInsights — Project Documentation

Project Title: AeroInsights — Airline Revenue Analytics Dashboard

---

1. Purpose

AeroInsights is a web-based analytics dashboard built to help airline revenue managers monitor key performance indicators in one place. The purpose is to give decision-makers a clear, real-time view of revenue trends, seat occupancy, forecast accuracy, and route-level performance — without needing to manually read through spreadsheets or reports. It is built as a React single-page application that fetches live data from an external API and presents it through interactive charts, tables, and insight cards.

---

2. Problem Statement

Airline revenue management involves tracking multiple metrics across dozens of routes simultaneously — total revenue, load factor, forecast gaps, and occupancy rates. Without a centralized dashboard, teams have to piece together information from separate reports, which is slow and error-prone. There is no easily accessible tool for students or small teams to visualize this data interactively. AeroInsights addresses this by building a lightweight, browser-based dashboard that aggregates and displays airline KPIs in a single interface, secured behind user authentication.

---

3. Goals

- Allow users to register and log in securely using LocalStorage-based authentication.

- Display five real-time KPI metrics fetched from an external API and transformed into airline-relevant data.
- Visualize revenue trends using a line chart comparing actual vs forecast revenue across 8 months.
- Show seat occupancy breakdown using a donut pie chart with route-level status indicators.
- Compare monthly forecast against actual revenue using a grouped bar chart with a gap analysis table.
- Surface auto-generated insights and alerts categorized by type — success, warning, and informational.
- Ensure all module pages are protected and only accessible after login.
- Demonstrate all six course outcomes through the codebase with inline CO-labeled comments.

---

4. Scope

In Scope:

- User registration and login using LocalStorage (no backend server).
- Protected routing — Dashboard, Revenue, Load Factor, Forecast, and Insights pages are accessible only after login.
- Live API integration using Axios with dummyjson.com — two endpoints fetched simultaneously using Promise.all.
- Data transformation using ES6 array methods (.map, .reduce, .filter) to convert product/user data into airline KPIs.
- Three Recharts visualizations — LineChart, PieChart (donut), BarChart.
- Global state management using React Context API.
- Filter controls (route dropdown, date range) shared across module pages via Context.
- Logout functionality that clears LocalStorage and redirects to Home.
- Responsive layout that works at 768px screen width without horizontal scrolling.

Out of Scope:

- No real airline data or official API — dummyjson data is mapped and scaled to simulate airline metrics.
- No backend, database, or server-side logic — all data is client-side.

- No real payment, booking, or ticketing functionality.
- No user profile editing or password reset.
- No dark mode or theme switching.
- No unit testing or automated test suites.

---

5. Users

Primary User — Airline Revenue Analyst:
A professional or student who wants to monitor revenue performance, seat occupancy, and forecast accuracy across routes. They log in, view the dashboard summary, and navigate to specific module pages for deeper analysis.

Secondary User — Faculty Evaluator:
A professor or examiner reviewing the project to assess whether React concepts, ES6 features, API integration, routing, and state management are correctly implemented and documented.

Both users interact through a browser — no installation beyond running npm run dev is required.

---

6. Functional Requirements

ID	Requirement
1.	A new user can register with full name, username, and password. Credentials are stored in LocalStorage as a JSON array.
2.	A registered user can log in. Valid credentials set isLoggedIn = true in LocalStorage and redirect to /dashboard. Invalid credentials show an alert.
3.	All module pages (/dashboard, /revenue, /load-factor, /forecast, /insights) check LocalStorage on mount and redirect to /login if the user is not authenticated.
4.	On load, the app fetches data from two dummyjson endpoints using Axios and Promise.all, transforms the response using array methods, and stores the result in React Context.
5.	The Dashboard page displays 5 KPI cards (Total Revenue, Total Passengers, Load Factor, Total Flights, Net Profit) and 4 module summary banners.
6.	The Revenue page displays a LineChart with actual and forecast lines, 4 stat boxes, and a Top Routes table with 5 rows.

7.	The Load Factor page displays a donut PieChart with occupied vs unoccupied seats and a 7-row route occupancy table with status badges.
8.	The Forecast page displays a grouped BarChart comparing actual vs forecast for 8 months and a gap analysis table for completed months.
9.	The Insights page displays 6 insight cards with color-coded left borders (green for success, yellow for warning, blue for informational).
10.	The Filter Controls component displays a route dropdown and date input. Changing either updates shared Context state visible across all module pages.
11.	The Logout button in the Header removes isLoggedIn and username from LocalStorage and redirects to the Home page.
12.	If the API call fails, the app displays a fallback error state instead of crashing.

---

7. Non-Functional Requirements

ID	Requirement
1.	**Performance** — The app loads in under 3 seconds on a standard broadband connection. API calls run in parallel using Promise.all to minimize wait time.
2.	**Responsiveness** — All pages render without horizontal scrolling on screens 768px and wider. Grids use auto-fit minmax layouts that adapt to screen width.
3.	**Reliability** — If the external API is unreachable, the app falls back to a safe error state using try/catch/finally. The UI does not crash or display blank pages.
4.	**Maintainability** — Every file includes CO-labeled inline comments explaining which course outcome each code block satisfies. The folder structure separates pages, components, context, and data clearly.
5.	**Usability** — Form inputs give immediate feedback on validation failure via alert dialogs. Loading spinners are shown while API data is being fetched. Navigation is consistent across all pages through a shared Header component.
6.	**Security (client-side)** — Passwords are stored as plain strings in LocalStorage, which is acceptable for a client-side academic project. The app does not transmit credentials to any server.
7.	**Build system** — The project uses Vite as the bundler. npm run dev starts a local dev server, npm run build generates a production-ready dist/ folder suitable for deployment on Netlify or Vercel.

---

8. Assumptions

- The user has Node.js (v18 or above) installed locally to run npm install and npm run dev.
- The app assumes a stable internet connection is available for the dummyjson API call. If offline, the fallback error state is shown.
- LocalStorage is available in the user's browser. All modern browsers support this by default.
- dummyjson.com remains publicly accessible and returns data in the expected format (products array with price and stock fields, users array with id).
- The data returned by dummyjson does not represent real airline data. It is transformed and scaled purely to demonstrate API integration and data processing concepts.
- Users register and log in on the same browser and device. Since LocalStorage is browser-specific, credentials do not transfer between browsers or devices.
- No accessibility audit has been performed beyond semantic HTML usage (header, main, nav, footer tags).
- The project is intended for academic demonstration, not commercial or production deployment.

---

9. Acceptance Criteria

ID	Criteria	Pass Condition
1.	User Registration	A new user can sign up with full name, username, and password. Credentials are stored in LocalStorage under the key users as a JSON array. Duplicate usernames are rejected with an alert.
2.	User Login	Valid credentials redirect to /dashboard and set isLoggedIn = true in LocalStorage. Invalid credentials show an alert and stay on the login page.
3.	Protected Route	Directly accessing /dashboard, /revenue, /load-factor, /forecast, or /insights without login redirects the user to /login.
4	KPI Display	After login, Dashboard shows 5 KPI cards with values dynamically fetched from dummyjson and transformed — not hardcoded.
5	Revenue Page	LineChart renders with two lines (actual solid, forecast dashed). Top routes table shows 5 rows with route name, revenue, and contribution bar.

6	Load Factor Page	Donut PieChart renders with occupied and unoccupied seat segments. Route table shows 7 rows with status badges (Good / Average / Low).
7	Forecast Page	BarChart renders grouped bars for actual and forecast per month. Gap table shows correct "Above Target" or "Below Target" labels per month.
8	Insights Page	All 6 insight cards render with correct color-coded left borders — green for success, yellow for warning, blue for informational.
9	Filter Controls	Changing the route dropdown or date input updates the displayed filter values across all module pages via Context.
10	Logout	Clicking Logout removes isLoggedIn and username from LocalStorage and navigates to the Home page. Module nav links disappear from the Header.
11	Responsiveness	All pages render without horizontal scrolling on a 768px wide screen. Grids reflow to single column on smaller widths.
12	API Fallback	If dummyjson is unreachable, the app shows "N/A" in KPI cards and a warning insight card instead of crashing or showing a blank screen.